Module 11 Lab: Deploying a Computer Vision Model as an API

Welcome to the final and most practical lab of the course! So far, you have learned how to build and train computer vision models. But how do you make your model useful to others? In this lab, you will learn how to **deploy** a model as an **API (Application Programming Interface)**. This will allow other applications to use your model to make predictions.

What you'll learn:

- What it means to deploy a model.
- What an API is and why it's useful.
- How to use the FastAPI framework to create a simple API for your model.
- How to send a request to your API and get a prediction back.

Learning Objectives

By the end of this lab, you will be able to:

- **Explain** the concept of model deployment and its importance.
- **Create** a simple web API using FastAPI.
- **Integrate** a pre-trained computer vision model into a FastAPI application.
- **Deploy** a computer vision model as a local API.

1. Setup and Installation

We will need a few new libraries for this lab, including `fastapi` for creating the API and `uvicorn` for running it.

```
%pip install fastapi uvicorn python-multipart transformers torch
```

```
Requirement already satisfied: uvicorn in /usr/local/lib/python3.12/dist-packages (0.51.0)
Requirement already satisfied: python-multipart in /usr/local/lib/python3.12/dist-packages (0.0.32)
Requirement already satisfied: transformers in /usr/local/lib/python3.12/dist-packages (5.13.1)
Requirement already satisfied: torch in /usr/local/lib/python3.12/dist-packages (2.11.0+cu128)
Requirement already satisfied: starlette>=0.46.0 in /usr/local/lib/python3.12/dist-packages (from fastapi) (1.3.1)
Requirement already satisfied: pydantic>=2.9.0 in /usr/local/lib/python3.12/dist-packages (from fastapi) (2.13.4)
Requirement already satisfied: typing-extensions>=4.8.0 in /usr/local/lib/python3.12/dist-packages (from fastapi) (4.16.0)
Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/lib/python3.12/dist-packages (from fastapi) (0.4.2)
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12/dist-packages (from fastapi) (0.0.4)
Requirement already satisfied: click>=7.0 in /usr/local/lib/python3.12/dist-packages (from uvicorn) (8.4.2)
Requirement already satisfied: h11>=0.8 in /usr/local/lib/python3.12/dist-packages (from uvicorn) (0.16.0)
Requirement already satisfied: huggingface-hub<2.0,>=1.5.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (1.23.0)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from transformers) (2.0.2)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (26.2)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from transformers) (6.0.3)
Requirement already satisfied: regex>=2025.10.22 in /usr/local/lib/python3.12/dist-packages (from transformers) (2025.11.3)
Requirement already satisfied: tokenizers<=0.23.0,>=0.22.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (0.22.2)
Requirement already satisfied: typer in /usr/local/lib/python3.12/dist-packages (from transformers) (0.26.8)
Requirement already satisfied: safetensors>=0.8.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (0.8.0)
Requirement already satisfied: tqdm>=4.60 in /usr/local/lib/python3.12/dist-packages (from transformers) (4.67.3)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch) (3.29.7)
Requirement already satisfied: setuptools<82 in /usr/local/lib/python3.12/dist-packages (from torch) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch) (3.6.1)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from torch) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-packages (from torch) (2025.3.0)
Requirement already satisfied: cuda-toolkit==12.8.1 in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,nvtx]==12.8.1; platform_system == "Linux"->torch) (12.8.1)
Requirement already satisfied: cuda-bindings<13,>=12.9.4 in /usr/local/lib/python3.12/dist-packages (from torch) (12.9.7)
Requirement already satisfied: nvidia-cudnn-cu12==9.19.0.56 in /usr/local/lib/python3.12/dist-packages (from torch) (9.19.0.56)
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from torch) (0.7.1)
Requirement already satisfied: nvidia-nccl-cu12==2.28.9 in /usr/local/lib/python3.12/dist-packages (from torch) (2.28.9)
Requirement already satisfied: nvidia-nvshmem-cu12==3.4.5 in /usr/local/lib/python3.12/dist-packages (from torch) (3.4.5)
Requirement already satisfied: triton==3.6.0 in /usr/local/lib/python3.12/dist-packages (from torch) (3.6.0)
Requirement already satisfied: nvidia-cublas-cu12==12.8.4.1.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,nvtx]==12.8.1; platform_system == "Linux"->torch) (12.8.4.1)
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.8.90.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,nvtx]==12.8.1; platform_system == "Linux"->torch) (12.8.90)
Requirement already satisfied: nvidia-cufft-cu12==11.3.3.83.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,nvtx]==12.8.1; platform_system == "Linux"->torch) (11.3.3.83)
Requirement already satisfied: nvidia-cufile-cu12==1.13.1.3.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,nvtx]==12.8.1; platform_system == "Linux"->torch) (1.13.1.3)
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.8.90.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,nvtx]==12.8.1; platform_system == "Linux"->torch) (12.8.90)
Requirement already satisfied: nvidia-curand-cu12==10.3.9.90.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,nvtx]==12.8.1; platform_system == "Linux"->torch) (10.3.9.90)
Requirement already satisfied: nvidia-cusolver-cu12==11.7.3.90.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,nvtx]==12.8.1; platform_system == "Linux"->torch) (11.7.3.90)
Requirement already satisfied: nvidia-cusparse-cu12==12.5.8.93.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,nvtx]==12.8.1; platform_system == "Linux"->torch) (12.5.8.93)
Requirement already satisfied: nvidia-nvjitlink-cu12==12.8.93.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,nvtx]==12.8.1; platform_system == "Linux"->torch) (12.8.93)
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.8.93.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,nvtx]==12.8.1; platform_system == "Linux"->torch) (12.8.93)
Requirement already satisfied: nvidia-nvtx-cu12==12.8.90.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,nvtx]==12.8.1; platform_system == "Linux"->torch) (12.8.90)
Requirement already satisfied: cuda-pathfinder~=1.1 in /usr/local/lib/python3.12/dist-packages (from cuda-bindings<13,>=12.9.4->torch) (1.5.6)
Requirement already satisfied: hf-xet<2.0.0,>=1.5.1 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=1.5.0->transformers) (1.5.1)
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=1.5.0->transformers) (0.28.1)
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.12/dist-packages (from pydantic>=2.9.0->fastapi) (0.7.0)
Requirement already satisfied: pydantic-core==2.46.4 in /usr/local/lib/python3.12/dist-packages (from pydantic>=2.9.0->fastapi) (2.46.4)
Requirement already satisfied: anyio<5,>=3.6.2 in /usr/local/lib/python3.12/dist-packages (from starlette>=0.46.0->fastapi) (4.14.2)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->torch) (3.0.3)
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from typer->transformers) (1.5.4)
Requirement already satisfied: rich>=13.8.0 in /usr/local/lib/python3.12/dist-packages (from typer->transformers) (13.9.4)
Requirement already satisfied: idna>=2.8 in /usr/local/lib/python3.12/dist-packages (from anyio<5,>=3.6.2->starlette>=0.46.0->fastapi) (3.18)
Requirement already satisfied: certifi in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub<2.0,>=1.5.0->transformers) (2026.6.17)
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub<2.0,>=1.5.0->transformers) (1.0.9)
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich>=13.8.0->typer->transformers) (4.2.0)
```

2. Understanding Model Deployment and APIs

What is Model Deployment?

Model deployment is the process of taking your trained model and making it available for use in a production environment. This means that other people and applications can send data to your model and get predictions back. It's the final step in the machine learning lifecycle.

What is an API?

An **API (Application Programming Interface)** is a set of rules and protocols that allows different software applications to communicate with each other. In our case, we will create a web API that allows other applications to communicate with our model over the internet using standard HTTP requests.

Why use FastAPI?

FastAPI is a modern, fast (high-performance) web framework for building APIs with Python. It is very easy to learn and use, and it automatically generates interactive API documentation, which is a huge plus!

Part 1: Coded Demonstration

In this part, we will create a simple API that can classify an image using a pre-trained model from Hugging Face. Because we are in a notebook environment, we will write the code to a Python file and then run it.

```
# 1. Write the API code to a Python file

api_code = '''
from fastapi import FastAPI, File, UploadFile
from transformers import pipeline
from PIL import Image
import io

# Create the FastAPI app
app = FastAPI()

# Load the image classification pipeline
classifier = pipeline("image-classification", model="google/vit-base-patch16-224")

@app.get("/")
def read_root():
    return {"message": "Welcome to the Image Classification API!"}

@app.post("/classify/")
async def classify_image(file: UploadFile = File(...)):
    # Read the image file
    contents = await file.read()
    image = Image.open(io.BytesIO(contents))

    # Get the prediction
    prediction = classifier(image)

    return {"filename": file.filename, "prediction": prediction}
'''

with open("main.py", "w") as f:
    f.write(api_code)
```

2. Run the API

Now, we will run the API using `uvicorn`. This will start a local web server that listens for requests. We will run this in the background so we can continue to use the notebook.

Note: You may need to stop and restart the kernel after running this cell to run the client code in the next step.

```
import subprocess
import time

# Start the server in the background
server_process = subprocess.Popen(["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"])

# Wait a moment for the server to start
time.sleep(20)
```

3. Test the API

Now that the API is running, let's send it an image and see what it predicts!

```
import requests
from PIL import Image
import io

# Download an image to test
image_url = "https://images.unsplash.com/photo-1543466835-00a7907e9de1?ixlib=rb-4.0.3&ixid=MnwxMjA3fDB8MHxwaG90by1wYWdlfHx8fGVufDB8fHx8&auto=format&fit=crop&w=1074&q=80"
response = requests.get(image_url)
image_bytes = io.BytesIO(response.content)

# Send the image to the API
files = {"file": ("dog.jpg", image_bytes, "image/jpeg")}
response = requests.post("http://localhost:8000/classify/", files=files)

# Print the prediction
print(response.json())
```

```
{'filename': 'dog.jpg', 'prediction': [{'label': 'beagle', 'score': 0.6580893397331238}, {'label': 'English foxhound', 'score': 0.2638916075229645}, {'label': 'Walker hound, Walker foxhound', 'score': 0.049863215535879135}, {'label': 'Brittany spaniel', 'score': 0.0036589589435607195}, {'label': 'EntleBucher', 'score': 0.0034517720341682434}]}
```

4. Stop the API Server

It's important to stop the server process when you're done.

```
server_process.terminate()
```

Part 2: Student Challenge

Your challenge is to create a new API that uses the YOLO object detection model from a previous lab. This will require you to modify the API to handle object detection instead of image classification.

Your Task:

1. Create a new Python file called `object_detection_api.py`.
2. In this file, create a FastAPI application that uses the `yolov8n.pt` model to detect objects in an image.
3. The API should have an endpoint that accepts an image and returns a list of detected objects with their bounding boxes.
4. Run your new API and test it with an image.

```
%pip install -q ultralytics
print("Ultralytics installed.")
```

```
Ultralytics installed.
```

```
# --- ENTER YOUR CODE HERE ---

# 1. Write your object detection API code to a file
# ...

object_detection_api_code = '''
from fastapi import FastAPI, File, UploadFile
from ultralytics import YOLO
from PIL import Image
import io

app = FastAPI()

# Load the pre-trained YOLOv8 nano model
model = YOLO("yolov8n.pt")

@app.get("/")
def read_root():
  return {"message": "Welcome to the YOLO Object Detection API!"}

@app.post("/detect/")
async def detect_objects(file: UploadFile = File(...)):
  contents = await file.read()
  image = Image.open(io.BytesIO(contents)).convert("RGB")

  results = model(image, verbose=False)

  detections = []

  for result in results:
    if result.boxes is not None:
      for box in result.boxes:
        class_id = int(box.cls.item())
        confidence = float(box.conf.item())
        x1, y1, x2, y2 = box.xyxy[0].tolist()

        detections.append({
            "label": model.names[class_id],
            "confidence": round(confidence, 4),
            "bounding_box": {
                "x1": round(x1, 2),
                "y1": round(y1, 2),
                "x2": round(x2, 2),
                "y2": round(y2, 2)
            }
        })
  return {
      "filename": file.filename,
      "detections": detections
  }
'''

with open("object_detection_api.py", "w") as f:
  f.write(object_detection_api_code)

print("Created object_detection_api.py")

# 2. Run your API
# ...

import subprocess
import time

detection_server_process = subprocess.Popen(
    [
        "uvicorn",
        "object_detection_api:app",
        "--host",
        "0.0.0.0",
        "--port",
        "8001"
    ]
)

time.sleep(8)

print("Object detection API started.")

# 3. Test your API
# ...

import requests
import io

image_url = "https://images.unsplash.com/photo-1543466835-00a7907e9de1?auto=format&fit=crop&w=1074&q=80"

image_response = requests.get(image_url, timeout=30)
image_response.raise_for_status()

image_bytes = io.BytesIO(image_response.content)

files = {
    "file": ("dog.jpg", image_bytes, "image/jpeg")
}

response = requests.post(
    "http://localhost:8001/detect/",
    files=files,
    timeout=120
)

print("Status code:", response.status_code)
print(response.json())

# 4. Stop your API
# ...

detection_server_process.terminate()
detection_server_process.wait()

print("Object detection API stopped.")
```

```
Created object_detection_api.py
Object detection API started.
Status code: 200
{'filename': 'dog.jpg', 'detections': [{'label': 'dog', 'confidence': 0.9065, 'bounding_box': {'x1': 197.73, 'y1': 100.95, 'x2': 810.55, 'y2': 804.57}}]}
Object detection API stopped.
```

Reflective Questions

Please answer the following questions in a new Markdown cell below.

1. Why is it better to deploy a model as an API instead of just having it as a script on your computer? What are the advantages?

- Deploying a model as an API allows other applications and users to access it without needing the original code or machine learning environment. This maked the model easier to integrate into websites, mobile apps, and other software. APIs also make it simpler to update the model in one place while allowing multiple users to access the latest version.

2. FastAPI automatically creates documentation for your API. How could this be useful for a team of developers working on a project?

- Automatic documentation helps developers understand how to use the API without reading all of the source code. It clearly shows the available endpoints, required inputs, and expected outputs. This makes collaboration easier, reduces confusion, and allows team members to test the API more efficiently.

3. Our API was deployed locally on our computer. What are some of the challenges you would face if you wanted to deploy this API to the cloud so that anyone in the world could use it? (Think about scalability, cost, security, etc.)

- Deploying the API to the cloud would require handling more users, which means the server must be able to scale when traffic increases. There are also costs for cloud computing, storage, and network usage. In addition, security becomes much more important because the API must be protected against unauthorized access, cyberattacks, and misuse.

4. What is MLOps and how does it relate to model deployment? Why is it an important field in AI?

- MLOps is the practice of managing the entire machine learning lifecycle, including training, testing, deployment, monitoring, and updating models. It helps ensure that models continue to perform well after they are deployed and makes it easier to maintain and improve them over time. MLOps is important because it allows AI systems to be reliable, scalable, and easier to manage in real-world applications.

Submission Instructions

1. Complete the **Student Challenge** section with your object detection API.
2. Answer the **Reflective Questions** in a new Markdown cell.
3. Save your completed notebook (`.ipynb` file) and submit it.